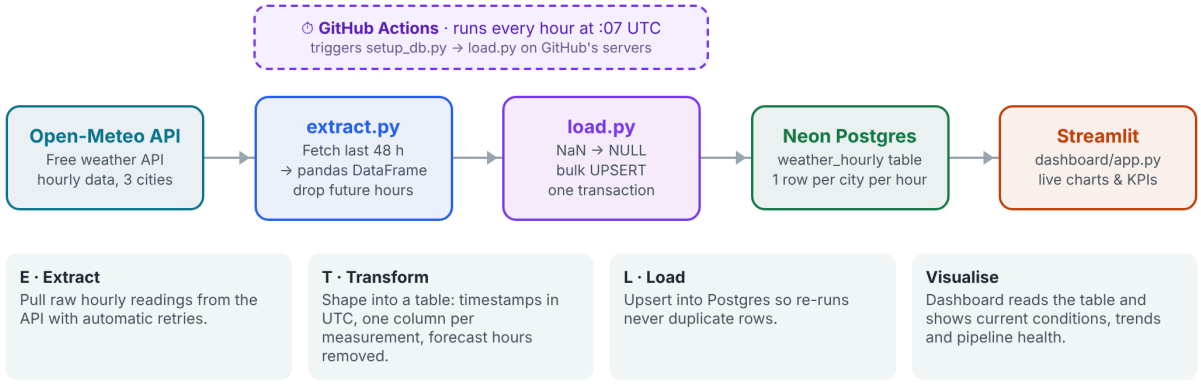
PROJECT REPORT · DATA ENGINEERING

# Live API Weather Pipeline

An automated hourly ETL pipeline with a PostgreSQL warehouse and a live Streamlit dashboard

### System architecture

How weather data travels from the API to the dashboard




**E · Extract**
Pull raw hourly readings from the API with automatic retries.

**T · Transform**
Shape into a table: timestamps in UTC, one column per measurement, forecast hours removed.

**L · Load**
Upsert into Postgres so re-runs never duplicate rows.

**Visualise**
Dashboard reads the table and shows current conditions, trends and pipeline health.

**Author:** Jenoah Gonsalves
**Email:** jenoah2004@gmail.com
**LinkedIn:** linkedin.com/in/jenoah-gonsalves-106113146
**Repository:** github.com/Jenzzzz7/Live-API-weather-pipeline
**Date:** October 2026

# Contents

1. Abstract
2. Introduction and objectives
3. Technology stack
4. System architecture
5. Implementation
6. Database design
7. Automation and scheduling
8. Dashboard
9. Testing and validation
10. Challenges and solutions
11. Results
12. Limitations and future work
13. Conclusion
14. References
15. Author

# 1. Abstract

This project builds an end-to-end data pipeline that collects live hourly weather observations for three Canadian cities (Halifax, Toronto and Vancouver) from the public Open-Meteo API. The data is transformed with pandas, loaded into a cloud PostgreSQL database (Neon), and visualised on an interactive Streamlit dashboard. GitHub Actions runs the pipeline automatically every hour. Because each run re-fetches a 48-hour window and loads it with an idempotent upsert, the system never stores duplicates and recovers automatically from failed runs. The project shows core data-engineering practices: ETL design, relational modelling, idempotent loading, scheduling, secrets management and data visualisation.

# 2. Introduction and objectives

Weather data is a good real-world source for a data pipeline: it changes every hour, it comes from an external API, and it is easy to sense-check. The aim was to build a system that keeps running on its own, not just a one-off script.

## Objectives

1. Extract hourly weather measurements for multiple cities from a live REST API.
2. Transform the raw response into a clean, consistent tabular format.
3. Load the data into a relational database without ever creating duplicate records.
4. Run the pipeline automatically every hour, without a dedicated server.
5. Recover automatically from temporary failures (network errors, API outages).
6. Present the data in a dashboard that also shows whether the pipeline is healthy.
7. Keep credentials secure and never commit them to source control.

## 3. Technology stack

| Layer | Technology | Purpose |
|---|---|---|
| Language | Python 3.11 | All pipeline and dashboard code |
| Data source | Open-Meteo Forecast API | Free hourly weather data, no API key needed |
| API client | openmeteo-requests, retry-requests | Decodes the binary FlatBuffers response; automatic retries with backoff |
| Transformation | pandas | Building the time series and tabular structure |
| Database | PostgreSQL on Neon (serverless) | Persistent storage, SQL querying |
| DB driver | psycopg2, SQLAlchemy | Batch upserts (pipeline); connection pooling (dashboard) |
| Scheduling | GitHub Actions (cron) | Hourly cloud execution with run logs |
| Visualisation | Streamlit | Interactive web dashboard |
| Config and secrets | python-dotenv, GitHub Secrets | Keeping credentials out of code |

## 4. System architecture

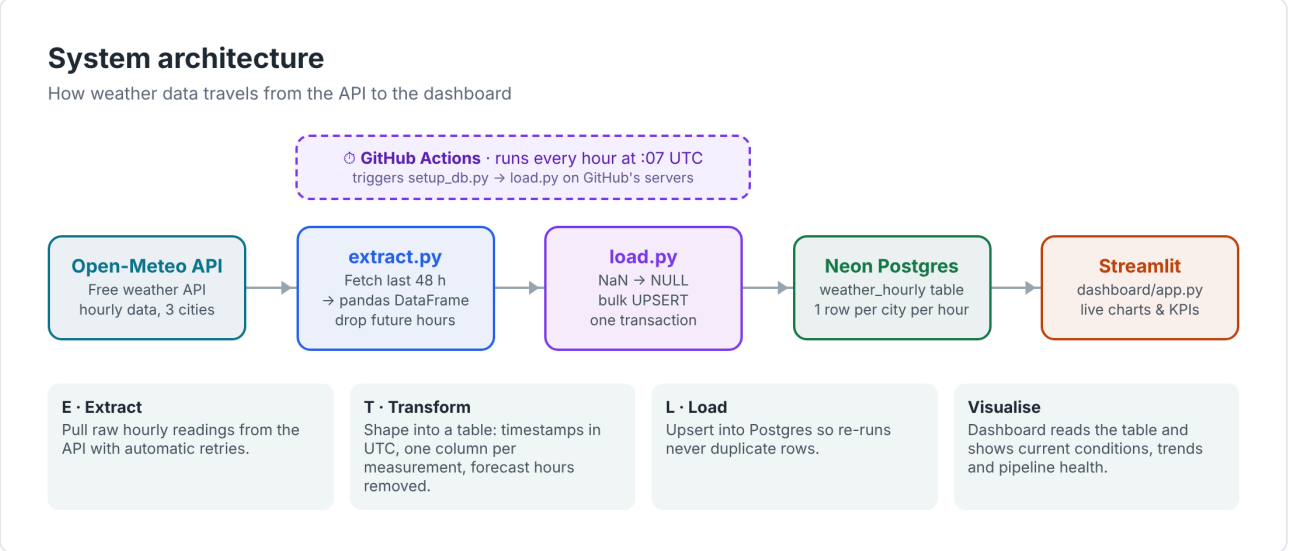



Figure 1: System architecture and the stages of the ETL process

The system follows the **Extract → Transform → Load** pattern. Each stage is a separate module with one responsibility:

- **extract.py** requests data from Open-Meteo and returns a pandas DataFrame (extract and transform).
- **load.py** runs the extraction and writes the result to PostgreSQL (load).
- **schema.sql / setup_db.py** define and create the target table.

- **pipeline.yml** schedules the job hourly on GitHub Actions.
- **dashboard/app.py** reads from the database and visualises the data.

Keeping the modules separate means each can be run and tested on its own. For example, `python extract.py` previews data without touching the database.

# 5. Implementation

## 5.1 Extraction

Cities are defined as a dictionary of name → (latitude, longitude), and the eight measurements as a list of Open-Meteo variable names: temperature, apparent temperature, relative humidity, precipitation, wind speed, wind gusts, surface pressure and cloud cover. For each city, the API is called with `past_days=2`, `forecast_days=1` and `timezone='UTC'`.

The HTTP session is wrapped with `retry(session, retries=5, backoff_factor=0.2)`. Failed requests are retried up to five times with increasing delays (exponential backoff), so brief network problems don't stop the pipeline.

## 5.2 Transformation

1. **Time index reconstruction.** The API returns a start time, end time and interval rather than individual timestamps. `pd.date_range` rebuilds the full hourly series in UTC.
2. **Positional variable mapping.** Each measurement is read with `hourly.Variables(i)` in the same order it was requested, so columns cannot be mismatched.
3. **City labelling.** A `city` column is added so all cities can be combined into one long-format table.
4. **Future-hour filtering.** Forecast rows later than the current time are removed, so only observed hours are stored.

## 5.3 Loading

The combined DataFrame is converted into Python tuples, with pandas `NaN` mapped to `None` (SQL `NULL`). All rows are sent in a single batch with `psycopg2.extras.execute_values`, inside one transaction: either every row is written, or the whole batch rolls back.

### Why re-runs never create duplicates: the upsert

Every hourly run re-fetches the last 48 hours. Postgres decides, row by row, whether to insert or update.

**Incoming rows (10:07 run)**

| CITY | OBSERVED_AT | TEMP | RESULT |
|---|---|---|---|
| Halifax | 08:00 | 7.9 | UPDATE |
| Halifax | 09:00 | 8.4 | UPDATE |
| Halifax | 10:00 | 9.1 | INSERT |

```
INSERT INTO weather_hourly (...)
VALUES %s
ON CONFLICT (city, observed_at)
DO UPDATE SET temperature_2m = EXCLUDED.temperature_2m,
              ... ,
              loaded_at = now()
```

INSERT hour not in the table yet → new row
UPDATE hour already stored → overwrite with latest values

Result: running load.py 1 time or 100 times gives the same table. This property is called **idempotency**.

Figure 2: The upsert decides, row by row, whether to insert or update

The load uses a PostgreSQL **upsert**: `INSERT … ON CONFLICT (city, observed_at) DO UPDATE`. New hours are inserted. Hours already stored are updated with the latest values, which also captures any revisions the weather service publishes. This makes the load **idempotent**: running it any number of times produces the same table.

## 6. Database design

### Database table: `weather_hourly`

One row per city per hour. The pair (city, observed_at) is the primary key.

| COLUMN | TYPE | MEANING | UNIT |
|---|---|---|---|
| **city** PK | TEXT | City name | – |
| **observed_at** PK | TIMESTAMPTZ | Hour the reading is for | UTC |
| temperature_2m | REAL | Air temperature, 2 m above ground | °C |
| apparent_temperature | REAL | "Feels like" temperature | °C |
| relative_humidity_2m | REAL | Relative humidity | % |
| precipitation | REAL | Rain + snow in that hour | mm |
| wind_speed_10m | REAL | Wind speed, 10 m above ground | km/h |
| wind_gusts_10m | REAL | Strongest gust | km/h |
| surface_pressure | REAL | Air pressure at the surface | hPa |
| cloud_cover | REAL | Sky covered by cloud | % |
| loaded_at | TIMESTAMPTZ | When the pipeline last wrote this row | UTC |

Figure 3: Structure of the weather_hourly table

- **Composite primary key (city, observed_at)** enforces one row per city per hour and is the constraint the upsert relies on.
- **TIMESTAMPTZ in UTC** avoids time-zone and daylight-saving errors across three time zones.
- **REAL** (4-byte float) gives enough precision for weather values while keeping storage small.
- **loaded_at** records when each row was last written. The dashboard uses it to monitor freshness.

- **Index on observed_at (descending)** keeps "latest readings" queries fast as data accumulates.
- **IF NOT EXISTS** makes the schema script safe to run on every scheduled run.

## 7. Automation and scheduling

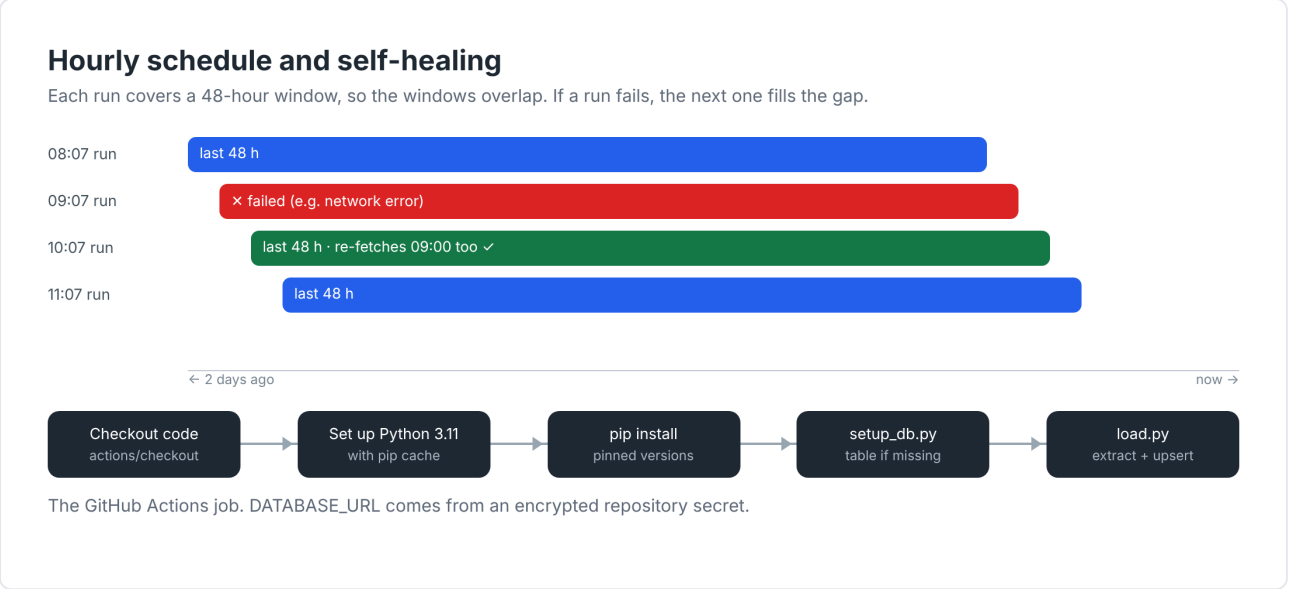



Figure 4: Overlapping 48-hour windows give automatic recovery; the GitHub Actions job steps

A GitHub Actions workflow runs at minute 7 of every hour (`cron: '7 * * * *'`). The small offset gives the API time to publish the newest hour. Each run provisions a fresh Ubuntu machine, installs Python 3.11 and the pinned dependencies, runs `setup_db.py`, then runs `load.py`. The connection string is passed in from an encrypted repository secret and never appears in code or logs. A `workflow_dispatch` trigger also allows manual runs.

> **Self-healing design:** each run covers the previous 48 hours, so consecutive runs overlap by 47 hours. If a run fails, the next successful run back-fills the missing hour automatically. No manual repair is needed for outages shorter than two days.

# 8. Dashboard

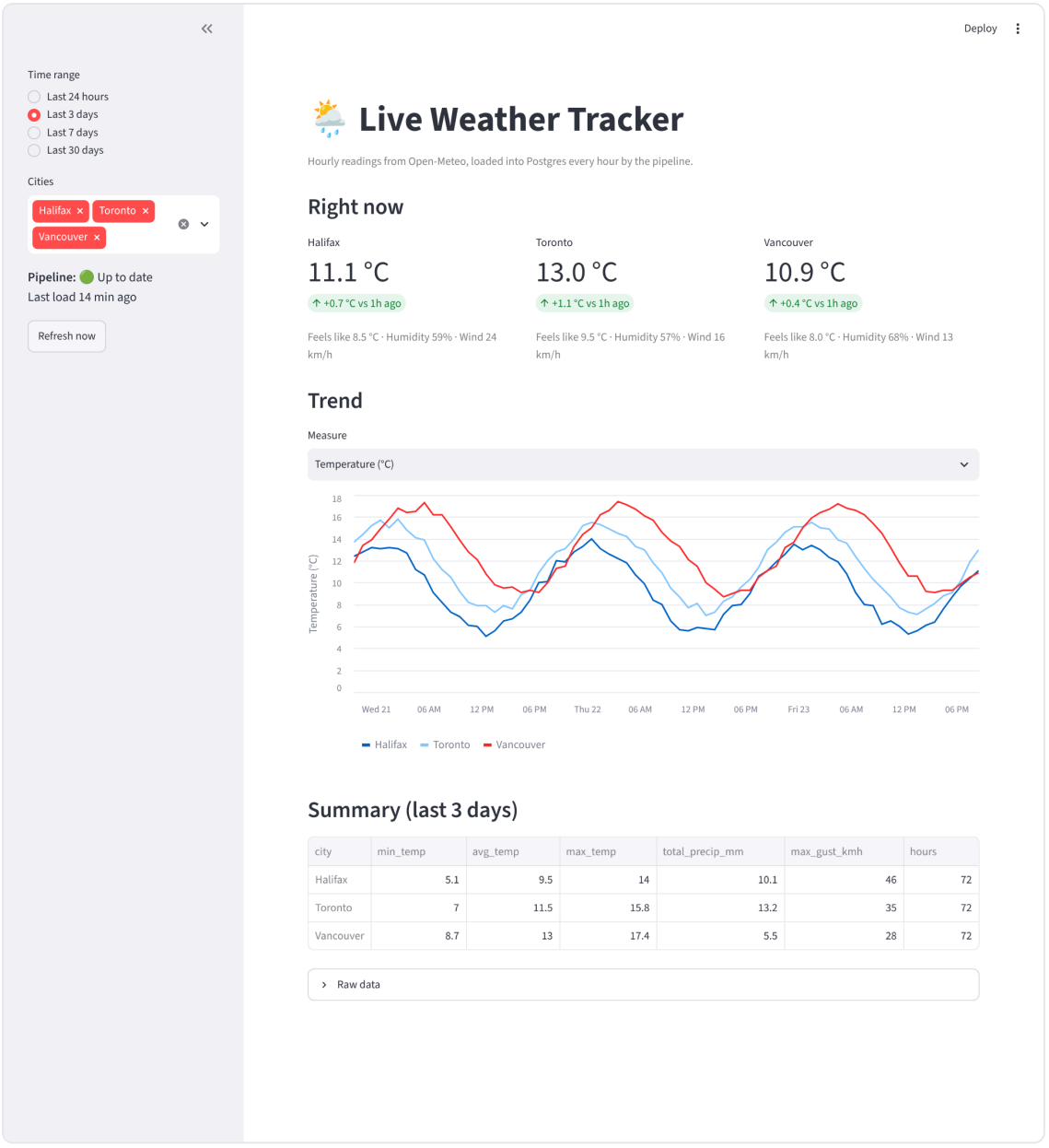



Figure 5: The Streamlit dashboard (shown with sample data)

- **Right now:** latest temperature per city with the change since the previous hour, plus feels-like, humidity and wind.
- **Trend:** line chart of any selected measurement over time, one line per city.
- **Summary:** minimum, average and maximum temperature, total precipitation, peak gust and hours recorded.
- **Pipeline health:** a green or red indicator based on the age of the most recent `loaded_at`. It turns red if no data has arrived in 90 minutes.
- **Filters:** time range (24 hours to 30 days) and city selection.

Query results are cached for 10 minutes (`st.cache_data(ttl=600)`) and the connection pool is created once (`st.cache_resource`), which keeps the database load low. Queries use bound parameters rather than string formatting, which prevents SQL injection.

## 9. Testing and validation

| Test | Method | Result |
|---|---|---|
| Schema creation is repeatable | Ran `setup_db.py` twice against a local PostgreSQL 16 instance | Pass: second run made no changes, no errors |
| Load is idempotent | Loaded the same 150 sample rows twice | Pass: row count stayed at 150 |
| Existing rows update | Re-loaded the same hours with changed values | Pass: values overwritten, still 150 rows |
| Missing values | Injected NaN into the precipitation column | Pass: stored as NULL |
| Dashboard renders | Streamlit AppTest plus a headless-browser screenshot | Pass after fixing the driver issue (see section 10) |
| Dashboard edge cases | Changed the measure; deselected all cities | Pass: no errors, helpful message shown |
| End-to-end live run | Run against the real API and Neon database | Run locally and through GitHub Actions |

## 10. Challenges and solutions

| Challenge | Solution |
|---|---|
| Re-running the pipeline would duplicate data | Composite primary key plus `ON CONFLICT DO UPDATE` upsert |
| Missed runs would leave gaps | 48-hour overlapping fetch window that back-fills automatically |
| API responses give a time range, not timestamps | Rebuild the hourly index with `pd.date_range` from start, end and interval |
| Forecast hours mixed with observations | Filter out rows later than the current UTC time |
| Running hourly without a server | GitHub Actions cron schedule |
| Keeping the database password private | `.env` (git-ignored) locally; encrypted secrets in GitHub and Streamlit |
| SQLAlchemy 2.1 defaulted to a Postgres driver (psycopg 3) that wasn't installed, which crashed the dashboard | Found in testing; fixed by explicitly selecting the `postgresql+psycopg2` dialect |
| Unpinned library updates could break the scheduled job | Pinned exact dependency versions in `requirements.txt` |

## 11. Results

- A fully automated pipeline that adds a new hourly reading for each of three cities (72 rows per day) with no manual work.
- A deduplicated, self-repairing dataset that grows over time and is ready for analysis.
- An interactive dashboard showing current conditions, trends, summaries and pipeline health.
- A codebase that is documented and configurable: adding a city is a one-line change.

## 12. Limitations and future work

### Limitations

- Recent hours come from Open-Meteo's weather models rather than individual weather stations, so values may differ slightly from local station readings.
- GitHub pauses scheduled workflows after 60 days without repository activity.
- Gaps longer than 48 hours are not back-filled automatically.
- No automated unit tests run in CI yet.

### Future work

- Daily aggregate tables or views for long-term trend analysis.
- Data-quality checks (missing hours, out-of-range values) with alerts.
- Threshold alerts, for example wind gusts above 70 km/h, sent by email or Discord.
- `pytest` unit tests running in GitHub Actions before each load.
- A historical back-fill using Open-Meteo's archive API.
- Public deployment of the dashboard on Streamlit Community Cloud.

## 13. Conclusion

The project meets all of its objectives. It delivers a reliable, automated ETL pipeline that extracts live weather data, transforms it into a clean structure, and loads it idempotently into PostgreSQL, with a dashboard on top. Overlapping fetch windows, upserts and transactional loads make the system resilient to the failures common in real data pipelines. Along the way, the project covered practical skills in API integration, data modelling, SQL, scheduling, secrets management and visualisation.

## 14. References


1. Open-Meteo Weather Forecast API documentation: open-meteo.com/en/docs
2. PostgreSQL documentation, INSERT ... ON CONFLICT: postgresql.org/docs/current/sql-insert.html
3. GitHub Actions documentation, scheduled events: docs.github.com/actions
4. Streamlit documentation, caching: docs.streamlit.io

5. pandas documentation: pandas.pydata.org/docs
6. Neon serverless Postgres: neon.tech/docs

Weather data provided by Open-Meteo.com under the CC BY 4.0 licence.

## 15. Author

**Jenoah Gonsalves**
Email: jenoah2004@gmail.com
LinkedIn: linkedin.com/in/jenoah-gonsalves-106113146
GitHub: github.com/Jenzzzz7